Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71251210**
Nama Lengkap	**Gabriel Ekklesia**
Minggu ke / Materi	**09 / Manajeman File**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

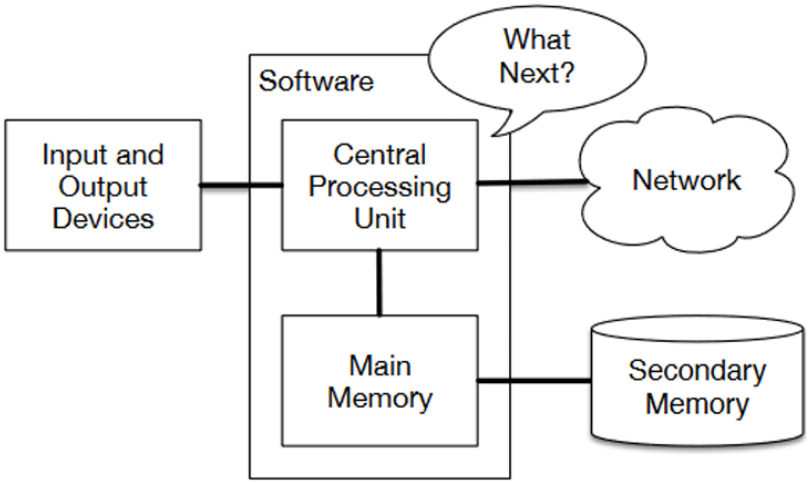
BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1

Pengantar File

Program yang berjalan membutuhkan memori utama (primary memory) sebagai tempat penyimpanan sementara. Selama program aktif, data akan disimpan di memori, tetapi akan hilang Ketika program selesai atau computer dimatikan karena sifatnya yang tidak permanen (volatile). Oleh sebab itu, memori utama tidak dapat digunakan untuk menyimpan data dalam jangka Panjang. Agar tetap tersimpan, diperlukan media penyimpanan sekunder (secpndary memory) yang bersifar permanen.



File disimpan pada secondary memory sehingga dappat digunakan untuk menyimpan data program dan tidak akan hilang meskipun computer dimatikan. Pada dasarnya, file merupakan kumpulan bit data yang tersimpan secara permanen dan berisi informasi yang saling berhubungan sebagai satu kesatuan. File dapat berupa berbagai jenis, seperti file system, file program (binary), file multimedia, maupun file teks. Selain itu, file juga memiliki beberapa properti, seperti nama file, ukuran, lokasi penyimpanan, pemilik, hak akses, serta waktu akses.

MATERI 2

Pengaksesan File

Untuk dapat mengakses file, langkah-langkah yang harus dilakukan adalah:

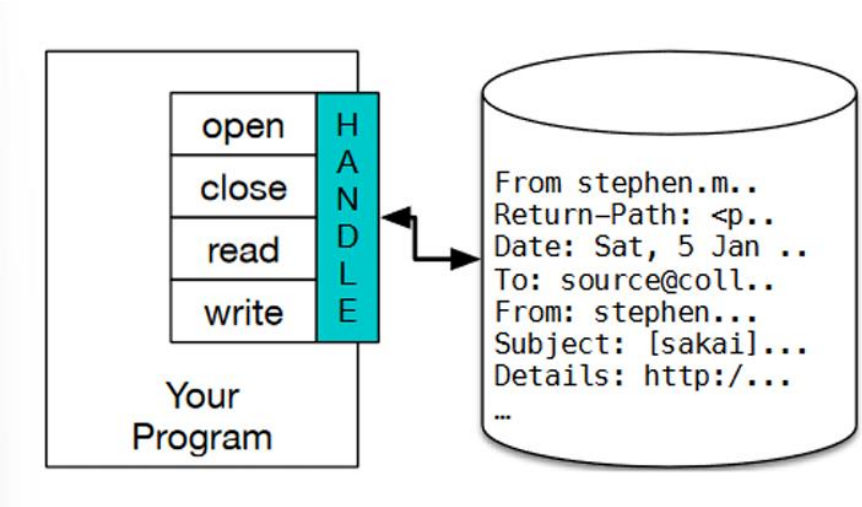
1. Menyiapkan file dan path yang akan diakses

2. Open file

3. Lakukan sesuatu dengan file tersebut, seperti ditampilkan (read) isinya atau diubah / ditulisi (write)

4. Close file

Gambaran handle file dapat dilihat pada gambar dibawah:




```
handle = open('mbox-short.txt')
print(handle)
```

Saat file dibuka, sistem akan menampilkan nama file, mode (r=read), dan encoding yang digunakan (umumnya UTF-8). Jika file tidak ditemukan, program akan menghasilkan error FileNotFoundError.

File teks biasanya terdiri dari baris-baris string, sehingga pembacaannya dilakukan per baris hingga mencapai EOF (End of File). Contoh yang ditampilkan menunjukkan isi file teks berupa header email, yang menegaskan bahwa data dalam file teks tersusun dalam bentuk baris-baris terstruktur. Contoh tampilan baris-baris dari sebuah file teks mbox-short.txt:

```
From stephen.marquard@uct.ac.za Sat Jan  5 09:14:16 2008
Return-Path: <postmaster@collab.sakaiproject.org>
Date: Sat, 5 Jan 2008 09:12:18 -0500
To: source@collab.sakaiproject.org
From: stephen.marquard@uct.ac.za
Subject: [sakai] svn commit: r39772 - content/branches/
Details: http://source.sakaiproject.org/viewsvn/?view=rev&rev=39772
...
```

MATERI 3

Manipulasi File

Cara membaca file pada Python adalah:

1. Siapkan file

2. Open file

3. Loop setiap baris pada file

4. Close file Cara pembacaan file adalah:

```
handle = open('mbox-short.txt')
count = 0
for line in handle:
    count = count + 1
print('Line Count:', count)
```

Catatan:

dibaca baris-perbaris Untuk mengatasi kemungkinan sistem membuka file yang besar sekali dalam satu waktu dan akhirnya hang / crash karena ukuran file yang sangat besar.

Cara menampilkan ukuran file teks dalam bytes, dapat digunakan fungsi len dari string.

```
handle = open('mbox-short.txt')
hasil = handle.read()
print("Ukuran: " + len(hasil) + "bytes")
print("Huruf dari belakang sendiri mundur 16 huruf adalah: " + hasil[-16::1])
```

Program membuka file mbox-short.txt, menghitung jumlah karakter (sebagai ukuran file dalam byte), lalu menampilkan 16 karakter terakhir dari isi file.

Catatan:

Perintah read() pada file sangat boros memory, sehingga jika ukuran file begitu besar maka lebih baik tidak menggunakan read(), melainkan menggunakan teknik loop seperti pada contoh di atas sebelumnya.

Selama looping, isi file bisa dimanipulasi, misalnya menampilkan hanya baris tertentu. Pada contoh file mbox, program dapat menampilkan hanya baris yang diawali dengan "Date:" dengan melakukan pengecekan pada setiap baris saat perulangan.

```
handle = open('mbox-short.txt')
count = 1
for line in handle:
    if line.startswith("Date:") and count <= 10:
        count += 1
        print(line)
```

Program di atas menghasilkan output 10 baris yang berupa tanggal saja.

Dobel newline terjadi karena setiap baris pada file sudah mengandung karakter newline (\n), kemudian perintah print() menambahkan newline lagi saat menampilkan output. Akibatnya, baris terlihat berjarak ganda. Untuk mengatasinya, dapat digunakan metode rstrip() pada setiap baris untuk menghapus newline, atau menggunakan print() dengan parameter end='' agar tidak menambahkan newline tambahan.

Contoh lain :

```
f = open('mbox-short.txt')

jumlah = 0

for baris in f:
    if "From" in baris:
        jumlah += 1

print("Jumlah baris yang mengandung :", jumlah)

f.close()
```

MATERI 4

Penyimpanan File

Contoh lengkap penulisan ke dalam file pada Python dapat dijelaskan sebagai berikut. Pertama, file dibuka menggunakan fungsi open() dengan mode "w" (write), misalnya fout = open('output.txt','w'), yang berarti file akan dibuat baru atau ditimpa jika sudah ada. Setelah file berhasil dibuka, data berupa string dapat langsung dituliskan ke dalam file tersebut menggunakan perintah write(<string>). Penulisan dapat dilakukan lebih dari satu kali sesuai kebutuhan, misalnya untuk menambahkan beberapa baris teks. Setelah semua data selesai ditulis, langkah terakhir yang harus dilakukan adalah menutup file dengan perintah close() agar semua perubahan tersimpan dengan baik. Dengan demikian, proses penulisan file terdiri dari tiga tahap utama, yaitu membuka file, menulis isi, dan menutup file kembali.

```
handle = open('output.txt','w')
tulisan = "teks ini akan dituliskan ke file\n"
handle.write(tulisan)
handle.close()
```

Contoh lain:

```
handle = open('mbox-short.txt', 'w')

handle.write("Baris pertama\n")
handle.write("Baris kedua\n")
handle.write("Baris ketiga\n")

handle.close()
```

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1

```python
f1 = open("Test1.txt", "r")
f2 = open("Test2.txt", "r")

data1 = f1.readlines()
data2 = f2.readlines()

i = 0
maks = max(len(data1), len(data2))

while i < maks:
    if i < len(data1):
        a = data1[i].strip()
    else:
        a = ""

    if i < len(data2):
        b = data2[i].strip()
    else:
        b = ""

    if a != b:
        print("beda di baris", i+1)
        print("file1:", a)
        print("file2:", b)
        print()

    i += 1

f1.close()
f2.close()
```

Dengan menggunakan metode split() untuk memisahkan isi baris serta perulangan yang didasarkan pada jumlah baris, program dapat membandingkan setiap baris dari file pertama dengan file kedua secara sistematis. Setiap baris yang telah dipisahkan kemudian diproses dan dibandingkan satu per satu. Melalui pendekatan ini, dapat ditentukan apakah isi suatu baris pada file pertama sama atau berbeda dengan baris yang bersesuaian pada file kedua.

SOAL 2

```
= open("Test3.txt","r")
= a.readlines()
lai = 0
r i in range (len(b)):
  print ()
  temp = b[i].strip().split("||")
  soal = temp[0].capitalize().strip()
  jawab = temp[1].lower().strip()
  print (soal)
  jawab1 = input("\nSilahkan masukan jawaban anda\n>> ").lower().strip()
  if jawab1 == jawab:
      print ("Jawaban anda benar")
      nilai += 20
  else:
      print ("Jawaban anda salah")
int (f"\nNilai anda {nilai}")
```

Dengan menggunakan metode split() untuk memisahkan data pada setiap baris file berdasarkan tanda ||, program dapat mengambil bagian soal dan jawaban secara terpisah. Selanjutnya, perulangan for yang didasarkan pada jumlah baris file digunakan untuk membaca setiap data secara berurutan. Setiap baris diproses satu per satu, kemudian jawaban pengguna dibandingkan dengan jawaban yang terdapat pada file. Melalui cara ini, program dapat menentukan apakah jawaban yang dimasukkan benar atau salah, lalu menghitung nilai sesuai jumlah jawaban yang benar.

LINK GITHUB: https://github.com/gilbetjuliesa-crypto/71251210_Gabriel_Ekklesia.git